Restaurant Billing & Management SaaS

Software Requirements Specification (SRS) | Developer Handoff | Version 1.0 | 12 October 2026

Status: Initial requirements for estimation, architecture, design, and implementation. Items marked “To confirm” require product-owner decisions before development.

1. Product vision and scope

Build a multi-tenant, cloud-based restaurant billing and management platform sold under Standard, Professional, and Premium subscriptions. Windows and Android are the initial supported operating systems. A central web-based Super Admin portal controls restaurant onboarding, licenses, administrator accounts, security, and auditing. Each restaurant sees only features permitted by its active subscription.

2. Actors and permission boundaries

Actor	Scope / core permissions
Platform Super Admin	Manage platform admins, restaurants, licenses, suspensions, MFA reset authority, platform audit logs.
Platform Admin	Delegated platform permissions; cannot exceed granted authority.
Restaurant Owner / Franchise Owner	View and manage own restaurant(s), authorized staff, operations, and reports.
Restaurant IT Admin	Configure restaurant users, devices, menu submissions, and technical settings within assigned permissions.
Cashier	Orders, table transfers, bills, taxes, tips, payments, authorized reversals.
Waiter	Assigned tables, order creation and updates, order tracking.
Chef	Kitchen order queue, recipe authoring, ingredient requirements, menu review.
Auditor	Read-only access to explicitly authorized records, reports, and logs.

Important: “Admin” must be split into platform-level and restaurant-level roles. Permissions are enforced on the server, not only by hiding buttons.

3. Platform administration (all plans)

3.1 Restaurant lifecycle

- Register a restaurant against a verified unique email address; generate a restaurant ID and tenant record.
- View/search/filter restaurant list by status, plan, license expiry, and owner.
- Activate, block, suspend, or reactivate restaurant accounts with recorded reason and actor.
- Keep historical records after suspension; define read-only/grace behavior for expired subscriptions (To confirm).

3.2 Licensing and subscription

- Associate each tenant with a plan, license start/end dates, payment status, permitted features, and configurable limits.
- Issue, renew, revoke, upgrade, or downgrade a license; record who made the change and why.

- Enforce entitlement checks in backend APIs and clients; avoid trusting a local license flag.
- Define payment collection method, billing cycles, proration, refunds, device/user/branch limits, and tax invoice rules (To confirm).

3.3 Platform admin and MFA

- Authorized Super Admins can invite/create other platform admins and assign granular roles.
- Require MFA for all platform administrators; support authenticator-app TOTP and recovery methods as an implementation choice.
- Allow only explicitly authorized admins to reset another admin's MFA, with step-up authentication, notification, and immutable audit trail.
- Protect against self-escalation, unauthorized role assignment, and unreviewed account recovery.

3.4 Audit and security logs

- Log authentication events, user creation, role changes, MFA reset, restaurant status, license changes, menu changes, table transfers, reversals, payments, and inventory adjustments.
- Store actor ID, tenant/branch, action, object type/ID, before/after values when appropriate, timestamp (UTC), device/session and reason.
- Audit records must be append-only for ordinary users; restrict access and establish retention policy (To confirm).
- Do not log passwords, tokens, MFA secrets, or full sensitive payment details.

4. Standard plan requirements

4.1 Restaurant, staff, tables

- Restaurant profile management and cashier accounts.
- Create tables with number/name and seating capacity; show free/occupied status and active order.
- Move an open order to another table while preserving items, timestamps, and audit history; handle destination conflicts.
- Maintain waiter directory and associate waiter with an order to attribute optional tips.

4.2 Ordering and billing

- Create dine-in and takeaway orders; add/edit/remove items with quantity and price snapshots.
- Create menu categories and items with name, description, quantity/unit, selling price, image, availability, and applicable tax.
- Calculate line totals, taxes, optional discount (permission-controlled), tip, and grand total; support printed bills/receipts.
- Cancel/return/reverse an item with mandatory reason; record actor, time, affected amount, and any approval requirement (To confirm).
- Preserve completed bill history and issue traceable adjustments instead of silently overwriting paid invoices.

5. Professional plan requirements

Includes all Standard capabilities.

- Multiple floors, each with an independently configured table layout; cross-floor order/table transfers.
- Multiple concurrent cashier accounts with real-time shared order/table/bill state and duplicate-payment protection.
- Waiter accounts with Android access to assigned tables, order entry, modifications, and status.
- Cashier access on Windows and Android; role-specific screens and permissions.
- Local LAN operation: a designated Windows device or on-premise gateway hosts a local service/database; Android and Windows clients communicate over restaurant Wi-Fi when cloud internet is unavailable.

- Queue changes locally and synchronize to cloud after reconnection using stable IDs, idempotent requests, conflict detection, and retry status.
- Explicitly handle local server outage, Wi-Fi outage, device restart, and cloud re-sync; no guarantee of offline communication if the local network itself fails.

6. Premium plan requirements

Includes all Standard and Professional capabilities.

- Create franchise organization with multiple restaurant branches, branch-specific staff and settings, and consolidated management dashboard.
- Generate a consolidated monthly franchise subscription bill with branch-wise charge breakdown.
- Provide IT Admin, cashier, waiter, chef, owner, and read-only auditor accounts with branch-scoped RBAC.
- Configure staff shifts, break windows, and attendance/break records.
- Kitchen Display System (Windows/Android as suitable): incoming order queue, item quantities, instructions, accepted/preparing/ready status, and preparation timestamps.
- Menu change workflow: authorized cashier/IT Admin submits item or ingredient requirements; chef reviews and approves/rejects; publication follows configured approval policy.
- Chef creates and versions recipes with ingredient quantities, units, and preparation instructions.
- Maintain department/location-wise inventory, receipts, stock transfers, consumption, adjustments, low-stock alerts, and wastage.
- Track daily dish quantities, sales revenue, ingredient consumption, inventory movement, and stock lifecycle from receipt through use/disposal.

7. Functional workflows and acceptance criteria

ID / workflow	Minimum acceptance criterion
AC-01 Tenant onboarding	Creating a verified-email restaurant produces an isolated tenant and a plan-bound license.
AC-02 License enforcement	Expired/blocked tenant cannot access disallowed actions; behavior follows approved expiry policy.
AC-03 MFA reset	Unauthorized admins cannot reset MFA; authorized reset is step-up verified and logged.
AC-04 Table transfer	An open order moves once to an available table; no lost or duplicated items.
AC-05 Item reversal	A reversal without a reason is rejected; accepted reversal is traceable and adjusts totals.
AC-06 Tip attribution	Tip is assigned to selected responsible waiter and visible in tip records.
AC-07 Concurrent cashiers	Two terminals see consistent order state; repeated payment requests do not double-charge.
AC-08 Offline LAN	With WAN disconnected but LAN operational, waiter order reaches cashier; reconnection syncs it once.
AC-09 KDS	Chef receives submitted order, updates preparation state, and staff see current status.
AC-10 Recipe inventory	Approved recipe consumption reduces mapped ingredient stock with auditable movement.
AC-11 Auditor	Auditor can view authorized records but cannot change them via UI or direct API.
AC-12 Franchise invoice	Monthly franchise invoice shows branch-level and consolidated subscription charges.

8. Suggested domain entities (initial model)

PlatformAdmin, PlatformRole, Permission, MFAEnrollment, RestaurantTenant, Franchise, Branch, SubscriptionPlan, License, PaymentRecord, User, UserRoleAssignment, Device, Floor, Table, MenuCategory, MenuItem, MenuImage, TaxRule, Recipe, RecipeIngredient, Order, OrderLine, OrderStatusHistory, TableTransfer, Bill, BillAdjustment, Payment, TipAllocation, StaffShift, BreakRecord, KitchenTicket, InventoryDepartment, InventoryItem, StockMovement, SupplierReceipt, AuditEvent, SyncOutbox, SyncConflict.

Every restaurant-scoped record must carry tenant_id; branch-specific records also carry branch_id. Financial records need stable IDs, immutable histories, and clearly defined status transitions.

9. Non-functional and architecture requirements

- Cloud service with API-based clients, secure database, backups, monitoring, and tenant isolation.
- Windows and Android application support; browser-based platform admin portal is acceptable.
- TLS for cloud connections; secure LAN pairing/authentication and encryption where feasible.
- Password hashing, session/token expiry, MFA, least privilege, rate limiting, and secrets management.
- Reliable local-first persistence for Professional/Premium, explicit sync status, idempotency keys, conflict-resolution policy, and reconciliation reports.
- Defined RPO/RTO, performance targets, maximum tenants/users/devices, image size limits, and retention policy (To confirm).
- Automated tests for entitlement checks, RBAC, financial calculations, offline reconnection, and multi-tenant isolation.

10. Proposed delivery phases

Phase	Scope / exit condition
1 — Foundation	Tenant model, authentication, RBAC, central admin, MFA, audit, plans and license enforcement.
2 — Standard MVP	Menu, table, dine-in/takeaway ordering, billing/taxes/tips, reversals, print, cashier UI.
3 — Professional	Floor layouts, multi-cashier concurrency, waiter Android app, local LAN service and cloud sync.
4 — Premium	Franchise/branch controls, chef KDS, approvals, recipes, inventory, scheduling, consolidated billing.
5 — Hardening	Security review, load tests, offline recovery tests, UAT, monitoring, backup/restore, release.

11. Open decisions requiring approval

- Subscription prices, durations, grace periods, plan limits, payment gateway, tax jurisdictions, and refund rules.
- Cash/payment modes, invoice numbering, fiscal compliance, discounts, split bills, partial payments, and receipt printer integrations.
- Whether Standard also needs offline billing; which Windows device acts as LAN server; behavior when it is switched off.
- Chef approval scope: recipe/ingredient requirements versus price changes; who has final publication authority.
- Inventory valuation method, stock unit conversions, batch/expiry tracking, and whether stock is deducted at order acceptance or preparation completion.
- Employee attendance source, break enforcement, and payroll integration (if any).
- MFA method, recovery policy, audit retention, data residency, and backup targets.
- Final UI designs, languages, deployment model, and device compatibility requirements.

12. Definition of done / handoff checklist

- Developer delivers architecture diagram, API contract, database schema, permission matrix, and screen map before full build.
- Each feature has automated tests and a demonstrated happy-path plus failure-path acceptance test.
- Critical operations (payment, reversal, license, MFA reset, sync) have audit logs and access checks.
- Professional offline test includes disconnect/reconnect and duplicate/conflict simulation.
- Deployment instructions, backup/restore procedure, monitoring, admin operations guide, and user documentation are supplied.

Document note: This is an implementation-ready requirements baseline, not a finalized legal, tax, or payment compliance specification. Open decisions must be resolved and signed off before their corresponding development work begins.